Capstone 7-A — Agent Evolution: Healthcare Pre-Auth

Build the SAME pre-authorization decision agent THREE times — first by hand with the raw API, then with the Agent SDK and Claude Code, then from a spec document. Same five clinical tools, same mock requests, same business question. Three radically different developer experiences.

A — Healthcare Pre-Auth | B — B2B Order Exception | C — UCC Risk Analyzer

Project Brief

THREE WAYS TO BUILD A HOUSE

Imagine you decide to build the same house three times. The first time, you fell every tree, mill every plank, and hammer every nail by hand. You learn exactly where the load-bearing walls go, why the joists are spaced 16 inches apart, and what happens when a sill plate is undersized. The build takes six months, but you understand every joint in the structure.

The second time, you order pre-cut lumber, pre-fab trusses, and use a nail gun. You finish in six weeks. The walls go up in the same places, but you are no longer wondering *why* — you are picking which tools to use and where to apply them. You build twice as fast and the structure is just as strong, but only because you already know what a structure should look like.

The third time, you hand a contractor a set of architectural drawings. Two weeks later, the house is done — built by people you never met to specifications you wrote in plain English. You are now an architect. The previous two builds taught you what to draw and what to leave to the crew.

This capstone is those three builds, compressed into one week. You will write the same pre-auth decision agent in raw API code, then with the Agent SDK and Claude Code, then as a 12-section spec that Claude Code reads and implements. By the end, you will know in your hands — not just in theory — why each layer of abstraction exists, what it costs, and when to reach for each one.

WHAT YOU'LL BUILD

You build a Pre-Authorization Decision Agent. It takes a pre-auth request — a procedure code (CPT), diagnosis code (ICD-10), member ID, and provider NPI — and reasons through five tool calls: looks up the payer's clinical criteria, checks whether the diagnosis meets medical necessity, verifies the provider's network status, pulls the member's benefit summary, and produces a structured determination (APPROVE / DENY / REQUEST_INFO) with a written clinical justification citing the policy section it relied on.

You build it three times:

- ITERATION 1 Raw API loop — `~250 lines`, `~3 hours`, hand-coded everything (M15B way)
- ITERATION 2 Agent SDK + Claude Code — `~120 lines`, `~2 hours` (M25–M26 way)
- ITERATION 3 Spec-driven — `~100 lines of spec`, `~1 hour` (production way)

HIPAA & PHI ARE NOT OPTIONAL

This scenario lets you exercise the same compliance constraints production health agents face: **PHI redaction in hooks** (member IDs and patient names should never enter logs verbatim), **audit trail requirements** (every tool call and determination must be persisted with a timestamp and case ID), and **determination explainability** (the rationale must cite specific clinical criteria sections, not "Claude said so"). All three iterations enforce the same rules — what changes is whether you write the redaction logic inline (Iter 1), as a hook (Iter 2), or specify it (Iter 3).

WHY IT MATTERS

Production teams do not pick "raw API vs SDK vs spec" in the abstract — they pick based on what the team already understands and what the problem requires. If you skip Iteration 1, you cannot debug Iteration 3 when the generated code does something weird. If you skip Iteration 3, you are 10x slower than the teams shipping agents in 2026. The point of building the same thing three times is that the *differences* teach you the trade-offs in a way no diagram ever will. You are not learning three different agents. You are learning three different *levels of abstraction*.

Three Iterations at a Glance

Same agent, same business question, same five tools, same mock data. Three layers of abstraction.

	Iter 1 — Raw API	Iter 2 — Agent SDK + Claude Code	Iter 3 — Spec-Driven
LINES YOU WRITE	~250 lines of Python	~120 lines (SDK + hooks + sessions)	~100 lines of spec (no agent code)
TIME TO BUILD	~3 hours	~2 hours	~1 hour
LOOP LIVES IN	Your `while True`	SDK's `query()`	Generated for you by Claude Code
DEBUG METHOD	`print()` in the loop + manual message inspection	Hooks as probes + Anthropic Console + Langfuse traces	Spec ↔ code comparison + tests + evals
KEY INSIGHT	Builds the mental model — every line is yours	Half the code; modular debug; sessions + hooks for free	The spec IS the documentation auditors read

The Three-Iteration Concept

Each iteration produces a working agent that solves the SAME pre-auth decision with the SAME five tools and SAME mock data. The agent's *output* is identical across all three. What changes is everything around the agent: the lines you wrote, the time you spent, the abstractions you used, and the way you debug when something breaks.

An agent is "a system prompt + tools + a loop." Iteration 1 makes you build all three from scratch. Iteration 2 keeps the system prompt and tools the same, but the SDK runs the loop for you. Iteration 3 keeps everything the same, but Claude Code writes the prompt, tools, AND loop from a spec you gave it. The agent is unchanged. You changed.

A COMMON MISUNDERSTANDING

"Iteration 3 is just better — why bother with the others?" *Because Iteration 3's generated code is not magic. When it produces a `verify_diagnosis_match` that does substring comparison instead of ICD-10 hierarchy traversal, you have to read the generated `tools.py`, find the bug, and fix it — either in the code or in the spec. Without Iteration 1's familiarity with what tool calls and message shapes look like, you cannot tell what the generated code is doing wrong. Iteration 3 is fast precisely because you can read its output.*

The Scenario — Pre-Auth Decision Agent

The agent takes a pre-authorization request and produces a determination: APPROVE / DENY / REQUEST_INFO with a clinical justification. The five-tool decision pattern mirrors what real prior-authorization software does, just with mock data instead of payer APIs.

BUSINESS QUESTION (USE THIS IN ALL THREE ITERATIONS)

"Should this pre-auth for knee replacement (CPT 27447) with diagnosis M17.11 (Unilateral primary osteoarthritis, right knee) be approved under Aetna for member ID A123456 with provider NPI 1234567890?"

TOOLS (5)

- `lookup_clinical_criteria(cpt_code, payer)` — medical necessity rules
- `verify_diagnosis_match(icd10_code, criteria)` — checks ICD-10 against criteria
- `check_network_status(provider_npi, payer)` — in-network / out-of-network
- `get_benefit_summary(member_id, cpt_code)` — coverage, copay, deductible
- `generate_determination(case_data)` — produces structured APPROVE/DENY/REQUEST_INFO

MOCK DATA SHAPE

- **15 pre-auth requests** across 5 procedures × 3 payers
- **Procedures:** MRI knee (CPT 73721), Knee replacement (27447), Specialty drug (J9035), Cardiac cath (93458), Physical therapy (97110)
- **Payers:** Aetna, UnitedHealth, BlueCross
- **Files:** `preauth_requests.json`, `clinical_criteria.json`, `provider_directory.json`, `benefits.json`

WANT A DIFFERENT DOMAIN?

Switch to Domain B (B2B Order Exception) or Domain C (UCC Risk Analyzer — default). The lab structure is identical; only the tools and data change.

Architecture Per Iteration

Each iteration produces the same logical agent (system prompt + 5 tools + a loop). What changes is the *physical* architecture — how much you own, how much the SDK owns, and how much is generated for you.

ITER 1 Raw API Loop

You own everything

agent.py *~250 lines, all hand-written*

```
while True:  the loop you wrote
client.messages.create(...)
if response.stop_reason == "end_turn": break
for block in response.content:
  validate_input · check_cost_cap · log · execute_tool · redact_phi · append · audit_log
```

tools.py	circuit_breaker.py	audit_log.jsonl
5 clinical tools you wrote	*you built*	*you rotate*

server.py (FastAPI) + Dockerfile — you wrote both

ITER 2 Agent SDK + Claude Code

You own the tools and config; SDK owns the loop

agent.py *~90 lines*

```
@tool("lookup_clinical_criteria", ...) ×5 functions
preauth_server = create_sdk_mcp_server(tools=[...])
OPTIONS = ClaudeAgentOptions(system_prompt=..., mcp_servers=..., hooks=...)
async for msg in query(prompt=..., options=OPTIONS): ...
```

The loop, message-passing, retries, and streaming live inside `claude-agent-sdk`. You do not write them.

Claude Code generated	claude-agent-sdk managed
`server.py · Dockerfile · tests · .claude/settings.json · hooks/*.py`	`query() loop · MCP transport · HookMatcher · resume tokens · streaming`

ITER 3 **Spec-Driven** *You own only the spec*

spec/agent-spec.md *~100 lines — the only file you write*

1. Overview
2. Configuration
3. Tools (5)
4. System Prompt
5. Hooks (PHI / tone / risk)
6. Sessions
7. Mock Data
8. API Wrapper
9. Deployment
10. Tests
11. Evaluation
12. File Structure

↓

`/generate-from-spec spec/agent-spec.md`

Claude Code reads the spec, generates ~18 files: `agent.py` · `sessions.py` · `mock_data/*.json` ×4 · `server.py` · `Dockerfile` · `.claude/settings.json` · `hooks/*.py` · `.claude/commands/*.md` · `tests/test_*.py` ×5 · `appendix/manual-loop.py` *(Iter-1 reference).*

Prerequisites

REQUIRED MODULES

- **M05 — Tool Use:** Tool definitions, tool_use blocks, the message loop
- **M12 — ReAct Agents:** Multi-step reasoning across the 5-tool decision flow
- **M15B — Build Complete Agent:** The whole Iter-1 mental model
- **M16–M17 — Guardrails & HITL:** Hooks, especially PHI redaction
- **M19 — Tracing:** Optional but useful for Iter 2 debugging
- **M21, M22B — Deployment:** FastAPI + Docker + Tier 1
- **M25, M26 — Claude Code & Hooks:** CLAUDE.md, slash commands, hooks API

IF YOU HAVE NOT DONE M15B / M26

Iteration 1 is a re-implementation of the M15B reference agent for the pre-auth scenario. Do M15B first if you have not built an agent from raw API calls. Iteration 2 leans on the `claude-agent-sdk` patterns taught in M26 (Hooks & Sessions & Agent SDK) — reach for it if `@tool` / `HookMatcher` / `ClaudeAgentOptions` feels unfamiliar.

TOOLS YOU'LL NEED INSTALLED

- Python 3.10+ with pip
- Claude Code CLI (`npm i -g @anthropic-ai/claude-code`) — for Iter 2 and 3
- Docker Desktop for the Tier 1 deployment
- `ANTHROPIC_API_KEY` environment variable
- Optional: a Langfuse account for Iter 2 tracing

SESSION 1

Iteration 1: Raw API Loop

Build the agent the M15B way. You write the loop, the validation, the PHI redaction, the audit, the sessions, and the deployment. Every line is yours. Every bug is yours to find.

`~3 hours` `~250 lines` `7 files` `0 abstractions`

Step 1: Setup & Mock Data

`15 min` `mock_data/*.json`

What & Why: Create the project folder, install `anthropic` + FastAPI, then build the four mock JSON files your tools will read. Mock data is what separates a "demo" agent from a "doesn't compile" agent — without realistic clinical criteria and ICD-10 mappings, every tool call returns garbage and you cannot tell if the loop is broken or the data is.

SETUP · MOCK_DATA/

SETUP

```
mkdir -p agent-iter1-raw/mock_data && cd agent-iter1-raw
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "anthropic≥0.40" "fastapi≥0.110" "uvicorn≥0.27" "pydantic≥2.0"
```

MOCK_DATA/

```
// mock_data/clinical_criteria.json (excerpt)
{
  "27447_AETNA": {
    "cpt_code": "27447",
    "procedure": "Total knee arthroplasty",
    "payer": "Aetna",
    "policy_id": "CPB-0660",
    "approved_indications": [
      "M17.10", "M17.11", "M17.12",
      "M17.30", "M17.31", "M17.32"
    ],
    "criteria": [
      "Symptomatic osteoarthritis confirmed by imaging",
      "Failed conservative therapy (NSAIDs, PT) for ≥ 6 months",
      "BMI < 40 OR documented exceptions"
    ]
  }
}

// mock_data/preauth_requests.json (excerpt)
[
  {
    "case_id": "PA-2025-0001",
    "member_id": "A123456",
    "provider_npi": "1234567890",
    "cpt_code": "27447",
    "icd10_code": "M17.11",
    "payer": "Aetna",
    "submitted": "2025-04-15"
  }
]

// mock_data/provider_directory.json (excerpt)
{
  "1234567890": {
    "npi": "1234567890",
    "name": "Dr. J. Smith",
    "specialty": "Orthopedic Surgery",
    "networks": ["Aetna", "BlueCross"]
  }
}

// mock_data/benefits.json (excerpt)
{
  "A123456_27447": {
    "covered": true,
    "deductible_remaining": 850.00,
    "copay": 250.00,
    "coinsurance_pct": 20,
    "prior_auth_required": true
  }
}
```

Build all four JSON files with realistic data: 15 pre-auth requests across 5 procedures (CPT 73721, 27447, J9035, 93458, 97110) and 3 payers (Aetna, UnitedHealth, BlueCross). Include 2 cases that should DENY (no medical necessity match), 2 that should REQUEST_INFO (out-of-network provider), and the rest that should APPROVE.

Run: `python -c "import json; print(len(json.load(open('mock_data/preauth_requests.json'))), 'requests')"`

```
15 requests
```

Checkpoint

You should see `15 requests`. If you see `0` or a JSON parse error, check that all four files are valid JSON.

Step 2: Define Tools as JSON Schema

`15 min` `tools.py`

What & Why: The Anthropic API needs every tool described as a JSON Schema object so Claude knows what arguments to pass. Pre-auth tools have stricter parameter shapes than most agents — ICD-10 codes follow a specific regex, NPIs are exactly 10 digits, CPT codes are 5-digit strings. Get the schema wrong and Claude either ignores the tool or passes the wrong types.

TOOLS.PY

TOOLS.PY

```
IMPORT json
FROM pathlib import Path

DATA = Path("mock_data")
CRITERIA = json.loads((DATA / "clinical_criteria.json").read_text())
PROVIDERS = json.loads((DATA / "provider_directory.json").read_text())
BENEFITS = json.loads((DATA / "benefits.json").read_text())

TOOLS = [
    {
        "name": "lookup_clinical_criteria",
        "description": "Get medical-necessity criteria for a CPT procedure under a specific payer.",
        "input_schema": {
            "type": "object",

    # … (full source in the desktop HTML) …

    IF name == "generate_determination":
        RETURN {"case_id": args["case_id"], "decision": args["decision"],
                "rationale": args["rationale"],
                "policy_citation": args.get("policy_citation", ""),
                "ts": "2025-04-15T12:00:00Z"}
    RAISE ValueError(f"Unknown tool: {name}")
```

Checkpoint

Run `python -c "from tools import execute_tool; print(execute_tool('check_network_status', {'provider_npi': '1234567890', 'payer': 'Aetna'}))"`. Expected: `{'npi': '1234567890', 'specialty': 'Orthopedic Surgery', 'in_network': True, 'payer': 'Aetna'}`.

Step 3: Build the While Loop

`45 min` `agent.py` `The CORE of Iter 1`

What & Why: This is the heart of the iteration — the agentic loop you will replace twice over. Write it once by hand and you will recognize what the SDK and Claude Code generate later. The system prompt is the most important part for pre-auth: Claude needs to know to call all 5 tools in the right order, never approve without a network check, and always cite a policy.

AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

AGENT.PY (PYTHON)

```python
IMPORT json, anthropic
FROM tools import TOOLS, execute_tool

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"
MAX_TURNS = 12
SYSTEM = """You are a pre-authorization decision agent for medical procedures.
For every case, follow this order:
1. lookup_clinical_criteria for the CPT + payer
2. verify_diagnosis_match for the patient's ICD-10
3. check_network_status for the provider + payer
4. get_benefit_summary for the member + CPT
5. generate_determination with APPROVE / DENY / REQUEST_INFO


    # … (full source in the desktop HTML) …

    RETURN text

IF __name__ == "__main__":
    q = ("Should pre-auth PA-2025-0001 (CPT 27447, ICD-10 M17.11, "
         "member A123456, provider NPI 1234567890, payer Aetna) be approved?")
    print(run_agent(q))
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — the raw API loop. Pre-auth decision flow.
IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT { TOOLS, executeTool } from "./tools.js";

CONST client = NEW Anthropic();
CONST MODEL = "claude-sonnet-4-6";
CONST SYSTEM = `You are a pre-authorization decision agent for medical procedures.
For every case, follow this order:
1. lookup_clinical_criteria for the CPT + payer
2. verify_diagnosis_match for the patient's ICD-10
3. check_network_status for the provider + payer
4. get_benefit_summary for the member + CPT
5. generate_determination with APPROVE / DENY / REQUEST_INFO


    # … (full source in the desktop HTML) …

      continue;
    }
    throw NEW Error(`Unexpected stop_reason: ${response.stop_reason}`);
  }
  throw NEW Error(`Agent exceeded ${maxTurns} turns without finishing`);
}
```

Run: `python agent.py`

Expected output (paraphrased — Claude generates fresh text each run):

```
Determination: APPROVE
Case: PA-2025-0001 | Member: A123456 | CPT: 27447 (knee replacement)
Diagnosis: M17.11 (right knee primary osteoarthritis) — matches Aetna policy CPB-0660
approved indications. Provider NPI 1234567890 confirmed in-network. Member benefit
verified: covered, $250 copay, $850 deductible remaining.

Rationale: All Aetna CPB-0660 medical-necessity criteria satisfied (symptomatic OA
confirmed, in-network provider, benefit verified). Approving per policy CPB-0660 v2024.
```

✅ **Checkpoint — Step 3**

You should see `Determination: APPROVE` with a rationale citing **policy CPB-0660**, in-network confirmation, and covered benefit. If the agent stops after only 1–2 tool calls, the system prompt is too vague — reinforce the 5-step order with an explicit "use ALL FIVE tools before generating the determination."

Troubleshooting

- **Agent returns DENY when it should APPROVE** → check that `verify_diagnosis_match` returns `match: true` for M17.11 against the CPB-0660 indications. Mock data must include M17.11 in the approved_indications list.
- **tool_use_id error on second turn** → the assistant turn must be appended to `messages` BEFORE the `tool_result` turn. Check the order in `_run_messages`.
- **Agent loops forever, hits MAX_TURNS** → the system prompt isn't telling it when to stop. Add: "Once you have called generate_determination, stop — do not search further."
- **Agent hallucinates a CPT code or member ID not in the question** → tighten the system prompt: "Use ONLY the CPT code, ICD-10 code, NPI, and member ID provided in the user's request. Do not invent values."

Step 4: Add Guardrails Manually (incl. PHI Redaction)

`30 min` `guardrails.py`

What & Why: A loop that does whatever Claude asks is not safe to ship in healthcare. Production pre-auth agents need at minimum: input validation (reject malformed CPT/NPI/ICD-10), **PHI redaction** (member IDs, names, DOBs scrubbed from logs), a cost cap, and a circuit breaker. You write all four by hand.

GUARDRAILS.PY

GUARDRAILS.PY

```
IMPORT re

# PHI patterns to redact from logs (NOT from the agent's working memory)
SSN_RE = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
PHONE_RE = re.compile(r"\b\d{3}-\d{3}-\d{4}\b")
MEMBER_ID_RE = re.compile(r"\b[A-Z]\d{6,9}\b")        # e.g. A123456
DOB_RE = re.compile(r"\b\d{4}-\d{2}-\d{2}\b")         # ISO date

CPT_RE = re.compile(r"^[0-9]{5}$|^[A-Z][0-9]{4}$")
NPI_RE = re.compile(r"^[0-9]{10}$")
ICD10_RE = re.compile(r"^[A-Z]\d{2}(\.\d{1,4})?$")

COST_LIMIT_TOKENS = 50_000
CIRCUIT_FAIL_THRESHOLD = 3

    # … (full source in the desktop HTML) …

CLASS CircuitBreaker:
    FUNCTION __init__(self): self.failures = 0
    FUNCTION record(self, ok):
        self.failures = 0 IF ok else self.failures + 1
        IF self.failures ⩾ CIRCUIT_FAIL_THRESHOLD:
            RAISE RuntimeError("Circuit breaker tripped — aborting")
```

Now wire the guardrails into `_run_messages` in `agent.py`. Replace your existing `_run_messages` with the version below. Note: `redact_phi` is called only when writing to the audit log (Step 5) — the unredacted member_id flows back to Claude so the agent can chain tool calls (lookup → verify → check_network → benefits → determination).

AGENT.PY (REPLACE _RUN_MESSAGES)

AGENT.PY (REPLACE _RUN_MESSAGES)

```
# Add to the imports at the top of agent.py, CircuitBreaker, COST_LIMIT_TOKENS   # NEW

_breaker = CircuitBreaker()        # NEW — module-level instance

FUNCTION _run_messages(messages):
    total_tokens = 0                # NEW — cost cap counter
    FOR turn IN range(MAX_TURNS):
        response = client.messages.create(
            model=MODEL, max_tokens=4096, system=SYSTEM,
            tools=TOOLS, messages=messages,
        )
        total_tokens += (response.usage.input_tokens
                         + response.usage.output_tokens)              # NEW
        IF total_tokens > COST_LIMIT_TOKENS:                          # NEW

    # … (full source in the desktop HTML) …

            messages.append({"role": "user", "content": tool_results})
            CONTINUE

        RAISE RuntimeError(f"Unexpected stop_reason: {response.stop_reason}")

    RAISE RuntimeError(f"Agent exceeded {MAX_TURNS} turns without finishing")
```

Test that the guardrails fire on bad input:

```
python -c "from agent import run_agent; print(run_agent('Pre-auth for CPT BAD123 ICD M17.11 NPI 12345 Aetna member A123456'))"
# Expected: agent attempts lookup_clinical_criteria(cpt_code='BAD123'),
# validate_input raises ValueError, error flows back as is_error: true,
# Claude either reformulates with a valid CPT or apologizes.
```

✅ **Checkpoint — Step 4**

The clean PA-2025-0001 query still produces APPROVE (same as Step 3). The "BAD123" query gets rejected at `validate_input` — you should see the agent reformulate or apologize. If `RuntimeError: Cost cap exceeded`, your loop is unbounded; verify the token sum is being checked after every `messages.create`.

⚠ **PHI REDACTION IS TRICKIER THAN IT LOOKS**

Notice we did NOT call `redact_phi` on the tool_result that flows back to Claude. This is critical: the agent NEEDS the unredacted `member_id` (e.g., `A123456`) to call `get_benefit_summary` in the next turn. If you redact too aggressively, the agent passes `[MEMBER_ID_REDACTED]` to `get_benefit_summary` and the lookup fails with "no benefit record".

PHI redaction happens in **Step 5**'s `append_audit` function — only when writing to the persistent audit log, not in the runtime message stream.

Troubleshooting

- `ImportError: cannot import name 'validate_input' from 'guardrails'` → `guardrails.py` isn't in the project folder, or the function name is misspelled.
- **Agent fails on `get_benefit_summary` with "no benefit record"** → you redacted the member_id in the tool_result. Don't — only redact in `append_audit`.
- **Circuit breaker trips on first call** → `_breaker.record(ok=False)` is in the wrong branch. It belongs only in the except, not after `execute_tool`.

Step 5: Add HIPAA Audit Logging

`15 min` `audit.py + agent.py wiring`

What & Why: HIPAA requires audit trails for any system that touches PHI. Every tool call needs a timestamped, redacted record: case_id, tool, redacted inputs, redacted output summary, token count. **Critical:** redaction happens here when writing to disk — the agent's runtime message stream stays unredacted so tool chaining works (per Step 4 callout).

Create a new file `audit.py` in the project folder:

AUDIT.PY

AUDIT.PY

```
IMPORT json, datetime
FROM guardrails import redact_phi

FUNCTION append_audit(tool_name, args, result, tokens,
                      case_id= "default"):
    rec = {
        "ts": datetime.datetime.utcnow().isoformat() + "Z",
        "case_id": case_id,
        "tool": tool_name,
        "input_redacted": redact_phi(json.dumps(args, default=str)),
        "output_summary": redact_phi(str(result))[:200],
        "tokens": tokens,
    }
    WITH open("audit_log.jsonl", "a") as f:
        f.write(json.dumps(rec) + "\n")
```

Now wire it into `_run_messages`. Add the import and the `append_audit` call right after the successful `execute_tool`:

AGENT.PY (ADDITIONS)

AGENT.PY (ADDITIONS)

```
# Add to the imports at the top of agent.py:
FROM audit import append_audit                    # NEW

# Inside _run_messages, in the try-block right after `result = execute_tool(...)`:
                        result = execute_tool(block.name, block.input)
                        _breaker.record(ok=True)
                        # Use the case_id from the user's first message (PA-YYYY-NNNN).
                        case_id = next(
                            (w FOR w IN messages[0]["content"].split() IF w.startswith("PA-")),
                            "default"
                        )
                        append_audit(                              # NEW
                            tool_name=block.name,
                            args=block.input,
                            result=result,
                            tokens=total_tokens,
                            case_id=case_id,
                        )
                        tool_results.append({...})
                        # ... rest of try-block unchanged ...
```

Run: `python agent.py`

Then inspect the audit file:

```
cat audit_log.jsonl   # macOS/Linux
type audit_log.jsonl  # Windows cmd
Get-Content audit_log.jsonl  # Windows PowerShell
```

Expected output (5 lines, one per tool call) — notice the member_id is redacted:


```
{"ts": "2026-05-09T12:01:14Z", "case_id": "PA-2025-0001", "tool": "lookup_clinical_criteria",
"input_redacted": "{\"cpt_code\": \"27447\", \"payer\": \"Aetna\"}", "output_summary": "{\"policy_id\":
\"CPB-0660\", \"approved_indications\": ...}", "tokens": 1842}
{"ts": "2026-05-09T12:01:18Z", "case_id": "PA-2025-0001", "tool": "verify_diagnosis_match",
"input_redacted": "{\"icd10_code\": \"M17.11\", \"criteria_id\": \"27447_AETNA\"}", "output_summary": "
{\"match\": true, ...}", "tokens": 2510}
{"ts": "2026-05-09T12:01:22Z", "case_id": "PA-2025-0001", "tool": "check_network_status",
"input_redacted": "{\"provider_npi\": \"1234567890\", \"payer\": \"Aetna\"}", "output_summary": "
```

```
{\"in_network\": true, ...}", "tokens": 3185}
{"ts": "2026-05-09T12:01:26Z", "case_id": "PA-2025-0001", "tool": "get_benefit_summary", "input_redacted":
"{\"member_id\": \"[MEMBER_ID_REDACTED]\", \"cpt_code\": \"27447\"}", "output_summary": "{\"covered\":
true, \"copay\": 250, ...}", "tokens": 3920}
{"ts": "2026-05-09T12:01:30Z", "case_id": "PA-2025-0001", "tool": "generate_determination",
"input_redacted": "{\"case_id\": \"PA-2025-0001\", \"decision\": \"APPROVE\", ...}", "output_summary": "
{\"case_id\": \"PA-2025-0001\", \"decision\": \"APPROVE\"}", "tokens": 4480}
```


✅ **Checkpoint — Step 5**

You should see **5 lines** in `audit_log.jsonl` (one per tool call). The `input_redacted` for `get_benefit_summary` should contain `[MEMBER_ID_REDACTED]` — NOT `A123456`. The agent's response (printed to stdout) should still APPROVE the case — if it returns a "member not found" error, you accidentally redacted the member_id in the runtime stream, not just the audit log.

Troubleshooting

- `ImportError: No module named 'audit'` → `audit.py` is in the wrong folder. Move it next to `agent.py`.
- **audit_log.jsonl shows literal member_id** `A123456` → either you forgot the `redact_phi` call inside `append_audit`, or your MEMBER_ID_RE regex doesn't match the format. Test it: `python -c "from guardrails import redact_phi; print(redact_phi('A123456'))"` should print `[MEMBER_ID_REDACTED]`.
- **Agent returns "no benefit record"** → you wrapped the tool_result in `redact_phi` before sending it back to Claude. Don't — only the audit write redacts.
- **case_id is "default" instead of "PA-2025-0001"** → the parsing logic doesn't handle your prompt format. Pass case_id explicitly via the user's first message.

Step 6: Multi-Turn Case Sessions

`15 min` `session.py`

What & Why: Clinical reviewers ask follow-ups on the same case: "What if the provider were out-of-network?" or "What about the same procedure but for a UnitedHealth member?" Iter 1 implements multi-turn case sessions by maintaining a per-case messages list, appending the new user message, and reusing the same `_run_messages` helper from Step 3. A sliding window keeps the list bounded.

Create a new file `session.py` in the project folder:

SESSION.PY

SESSION.PY

```
FROM agent import _run_messages

SESSIONS, list] = {}   # case_id → messages list
WINDOW = 24                      # 5 tool calls per turn × ~5 turns + buffer

FUNCTION chat(case_id, user_msg):

    msgs = SESSIONS.setdefault(case_id, [])
    msgs.append({"role": "user", "content": user_msg})
    answer, msgs = _run_messages(msgs)
    # Sliding window= msgs[-WINDOW:]
    RETURN answer
```

Try it — multi-turn from the Python REPL:

```
python -c "
FROM session import chat
print(chat('PA-2025-0001', 'Should pre-auth PA-2025-0001 (CPT 27447, ICD M17.11, member A123456, NPI 1234567890,
payer Aetna) be approved?'))
print('---')
print(chat('PA-2025-0001', 'What IF the provider NPI were 9999999999 (out-of-network) instead?'))
"
```

Expected behavior: the second call references the same case context (CPT, ICD, member) but with the changed NPI — should produce REQUEST_INFO or DENY because the provider is out-of-network. Without session continuity, the agent would have no idea what "the same case" means.

> ✅ **Checkpoint — Step 6**
>
> The second call references CPT 27447 / M17.11 (proving the prior context survived) and produces a DIFFERENT determination because the NPI changed. If the second call says "I need more information" without the original CPT/ICD context, the session isn't carrying over — verify `SESSIONS` is populated after the first call.

Troubleshooting

- `ImportError: cannot import name '_run_messages' from 'agent'` → you skipped Step 3's refactor of `agent.py`. Go back and split the loop into `_run_messages` + `run_agent`.
- **Each call starts a fresh case** → `SESSIONS` is module-level. If you're calling from separate Python processes (e.g., via `subprocess`), use a database or Redis instead.
- **Context window exceeded after a few turns** → `WINDOW = 24` may be too generous. Each tool call adds 2 messages (assistant + tool_result), so 5 tools = 10 messages per turn. Drop to 12 for shorter cases.

Step 7: Deploy as FastAPI + Docker

`20 min` `server.py + Dockerfile`

What & Why: Wrap the agent in an HTTP API. Same Tier-1 deployment as M22B.

SERVER.PY · DOCKERFILE

SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run_agent
FROM session import chat

app = FastAPI()
CLASS Q(BaseModel): question: str
CLASS C(BaseModel): case_id: str; message: str

FUNCTION health(): return {"status": "ok"}

FUNCTION preauth(q):
    TRY: return {"determination": run_agent(q.question)}
    CATCH: raise HTTPException(500, str(e))

FUNCTION chat_ep(c):
    RETURN {"answer": chat(c.case_id, c.message)}
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Also create `requirements.txt` at the project root:

```
anthropic≥0.40
fastapi≥0.110
uvicorn≥0.27
pydantic≥2.0
```

Run locally first (no Docker):

```
uvicorn server:app --reload --port 8000
# In another terminal:
curl localhost:8000/health
# Expected: {"status":"ok"}

curl -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 approvable?"}'
# Expected: {"determination":"Determination: APPROVE..."}
```

Then build and run with Docker:

```
docker build -t iter1-a .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter1-a
```

Troubleshooting

- `docker: command not found` → install Docker Desktop and confirm `docker --version` works.
- `OSError: [Errno 98] Address already in use` → port 8000 is taken. `--port 8001` for uvicorn or `-p 8001:8000` for docker.
- **Container starts but /preauth returns 500 with "ANTHROPIC_API_KEY not set"** → you forgot the `-e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY` flag on docker run.
- **HIPAA concern: audit_log.jsonl is on the container's writable layer** → in production, mount a volume: `-v $(pwd)/audit:/app/audit` and have `append_audit` write to `/app/audit/audit_log.jsonl`.

Iteration 1 Complete — End-to-End Verification

You should now have **10 files** in your project folder:

```
agent-iter1-raw/
|— agent.py            # _run_messages + run_agent
|— tools.py            # 5 pre-auth tool schemas + execute_tool
|— mock_data/
|-   |— preauth_requests.json     # 15 cases across 5 procedures × 3 payers
|-   |— clinical_criteria.json    # CPB-0660 etc.
|-   |— provider_directory.json   # 10 providers w/ networks
|-   |— benefits.json             # 30 member-procedure records
|— guardrails.py       # validate_input + redact_phi + CircuitBreaker
|— audit.py            # HIPAA-compliant append_audit
|— session.py          # multi-turn case sessions
|— server.py           # FastAPI handlers
|— Dockerfile | requirements.txt
```

Run the full Iter-1 acceptance test:

```
# 1. Single pre-auth (should APPROVE)
curl -s -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 (CPT 27447, ICD M17.11, member A123456, NPI 1234567890, Aetna)
approvable?"}' | python -m json.tool

# 2. Multi-turn case session (out-of-network what-if)
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"Should this be approved? CPT 27447, ICD M17.11, member A123456, NPI
1234567890, Aetna"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"What if NPI 9999999999 (out-of-network) instead?"}' | python -m
json.tool

# 3. Guardrail fires on bad CPT
curl -s -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is CPT BAD123 approvable?"}' | python -m json.tool

# 4. HIPAA: audit log shows redacted PHI
docker exec $(docker ps -q --filter ancestor=iter1-a) cat audit_log.jsonl | head -5
# Look for "[MEMBER_ID_REDACTED]" instead of literal "A123456"
```

You pass Iter 1 if: (1) /health returns ok; (2) PA-2025-0001 returns APPROVE with policy CPB-0660 cited; (3) the second /chat call returns DIFFERENT determination (REQUEST_INFO or DENY) because the NPI changed; (4) the BAD123 query gets reformulated by the agent; (5) `audit_log.jsonl` has redacted PHI but the agent's runtime stream still resolves member benefits correctly.

ITERATION 1 METRICS

- **Files:** 10 (agent.py, tools.py, mock_data/×4, guardrails.py, audit.py, session.py, server.py, Dockerfile, requirements.txt)
- **Lines you wrote:** ~250
- **Time:** ~3 hours
- **Abstractions used:** none — just `anthropic` Messages API and FastAPI

Debugging in Iteration 1: Print Statements + Manual Inspection

When the agent gives wrong output in Iter 1, you debug like you debug any Python program: by reading your own code. There is no abstraction between you and Claude.

A. Add a debug_turn() helper

Drop this into `agent.py` and call it after each `messages.create`:

PYTHON

PYTHON

```
FUNCTION debug_turn(turn_num, response, messages):
    print(f"\n═══ TURN {turn_num} ═══ stop_reason: {response.stop_reason}")
    FOR block IN response.content═ "tool_use":
            print(f"  TOOL: {block.name}({block.input})")
        ELIF block.type ═ "text":
            print(f"  TEXT: {block.text[:120]}...")
    print(f"  Tokens={response.usage.input_tokens} out={response.usage.output_tokens}")
```

B. Inspect messages list manually

The most common Iter-1 bug is malformed messages. Add `print(json.dumps(messages, default=str, indent=2))` before each `messages.create`. You should see strict alternation.

C. Common Iter-1 bugs and how to spot them

- **Wrong `tool_use_id` in tool_result** → 400 from API. Match the `id` on the `tool_use` block.
- **Forgot to append the assistant turn** → API complains about message order.
- **Loop never stops** → Claude keeps asking for tools (system prompt too vague), or you forgot to handle `end_turn`.
- **Domain-specific:** agent calls `verify_diagnosis_match` with `icd10_code = "M17"` instead of `"M17.11"`. The validation regex catches it. Check the system prompt's instruction to use the FULL ICD-10 code from the request.
- **Domain-specific:** agent APPROVES even when `check_network_status` returned `in_network: false`. The system prompt did not enforce the gating rule firmly enough. Strengthen with: "Never APPROVE if in_network is false — that case is REQUEST_INFO."

D. Debug exercise (do this before moving on)

In `tools.py`, change the `icd10_code` regex to be too strict (require 4 digits after the dot). Re-run the agent. You should see `validate_input` raise a `ValueError`, the agent receive the error in `tool_result.is_error`, and either retry with a different code or fail gracefully. Restore the regex and re-run. **This is the muscle memory that lets you debug Iter 3 generated code later.**

SESSION 2

Iteration 2: Agent SDK + Claude Code

Now you let the SDK run the loop, hooks handle PHI redaction and validation, sessions handle multi-turn, and Claude Code does most of the typing. Same agent. Half the lines. Different debugging.

`~2 hours` `~120 lines` `8 files` `SDK + hooks + sessions`

Step 8: Create CLAUDE.md via Claude Code

`10 min` `CLAUDE.md`

What & Why: CLAUDE.md is the project memory file Claude Code reads at every prompt. For pre-auth, this is also where you encode the decision-order rules so Claude Code can generate a system prompt that gets it right the first time.

CLAUDE CODE · CLAUDE.MD

CLAUDE CODE

```
mkdir agent-iter2-sdk && cd agent-iter2-sdk
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install "claude-agent-sdk≥0.2" "fastapi≥0.110" "uvicorn≥0.27" "pydantic≥2.0"
npm i -g @anthropic-ai/claude-code   # if not already installed
claude
> /init
```

CLAUDE.MD

```
# Agent: Pre-Authorization Decision Agent (Iteration 2)

## Stack
- Python 3.11+
- `claude-agent-sdk` (the official Agent SDK — NOT a wrapper around `client.messages.create()`)
- FastAPI + Docker for deployment
- Mock data in mock_data/*.json (4 files)

## File Layout
- agent.py            — query() entry point + 5 @tool-decorated MCP tools + create_sdk_mcp_server
- hooks/              — PreToolUse + PostToolUse hook scripts
- .claude/settings.json — hooks registration (matchers + commands)
- sessions.py         — multi-case session resume
- server.py           — FastAPI wrapper

    # … (full source in the desktop HTML) …

3. check_network_status(provider_npi, payer)
4. get_benefit_summary(member_id, cpt_code)
5. generate_determination(...)

Never APPROVE without successful match AND in-network AND covered.
REQUEST_INFO when missing data; DENY when criteria fail.
```

Step 9: Build Agent with @tool Decorators (claude-agent-sdk)

`15 min` `agent.py`

What & Why: The `claude-agent-sdk` lets you define tools as `@tool`-decorated async functions registered with an in-process MCP server. `query()` drives the loop. This is a real package — do NOT simulate it with `client.messages.create()`.

WHAT THE REAL SDK LOOKS LIKE

If you've used `client.messages.create()` in Iter 1, you might expect the SDK to be a thin `Agent` class wrapping it. It is not. The SDK is built around **MCP tools** + **an async `query()` generator** + **options/hooks via `ClaudeAgentOptions`**. Tools return `{"content": [{"type": "text", "text": ...}]}` (MCP shape), not bare Python values.

CLAUDE CODE PROMPT · AGENT.PY (PYTHON) · AGENT.TS (TYPESCRIPT)

CLAUDE CODE PROMPT

```
> Create agent.py using `claude-agent-sdk`. Define five pre-auth tools as
> @tool-decorated functions returning MCP-shaped {"content":[...]}:
> lookup_clinical_criteria, verify_diagnosis_match, check_network_status,
> get_benefit_summary, generate_determination. Wire them into a
> create_sdk_mcp_server, build ClaudeAgentOptions with the system prompt
> from CLAUDE.md, and expose run(question) driving query() and
> concatenating AssistantMessage text.
```

AGENT.PY (PYTHON)

```python
IMPORT json
FROM pathlib import Path
FROM claude_agent_sdk import (
    query, tool, create_sdk_mcp_server,
    ClaudeAgentOptions, AssistantMessage,
)

DATA = Path("mock_data")
CRITERIA = json.loads((DATA / "clinical_criteria.json").read_text())
PROVIDERS = json.loads((DATA / "provider_directory.json").read_text())
BENEFITS = json.loads((DATA / "benefits.json").read_text())

@tool("lookup_clinical_criteria",
      "Get medical-necessity criteria for a CPT + payer.",

    # … (full source in the desktop HTML) …

    FOR msg IN query(prompt=question, options=OPTIONS):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None):
                    parts.append(block.text)
    RETURN "\n".join(parts)
```

AGENT.TS (TYPESCRIPT)

```typescript
// agent.ts — @anthropic-ai/claude-agent-sdk version
IMPORT { query, tool, createSdkMcpServer } from "@anthropic-ai/claude-agent-sdk";
IMPORT { z } from "zod";
IMPORT * as fs from "fs";

CONST CRITERIA  = JSON.parse(fs.readFileSync("mock_data/clinical_criteria.json", "utf8"));
CONST PROVIDERS = JSON.parse(fs.readFileSync("mock_data/provider_directory.json", "utf8"));
CONST BENEFITS  = JSON.parse(fs.readFileSync("mock_data/benefits.json", "utf8"));

CONST lookupClinicalCriteria = tool(
  "lookup_clinical_criteria",
  "Get medical-necessity criteria for a CPT + payer.",
  { cpt_code: z.string(), payer: z.string() },
  (args) ⇒ {

    # … (full source in the desktop HTML) …

        IF ("text" in block) parts.push(block.text);
      }
    }
  }
  RETURN parts.join("\n");
}
```

WHAT JUST DISAPPEARED

You no longer write the message loop, the stop_reason check, the tool_result append, or JSON schema dicts. The 90-line raw loop from Iter 1 collapsed to one `async for msg in query(...)` (Python) / `for await` (TS). **Same five-tool decision flow, ~90 lines.**

Troubleshooting

- `ModuleNotFoundError: No module named 'claude_agent_sdk'` → `pip install "claude-agent-sdk ⩾ 0.2"` in your venv.

- `ImportError: cannot import name 'Agent' from 'anthropic'` → you're trying the old fictional API. The real SDK is `claude_agent_sdk`.
- Tool call rejected with "not allowed" → add the tool's `mcp__preauth__<name>` entry to `allowed_tools`.

Step 10: HIPAA Hooks via .claude/settings.json + HookMatcher

`20 min` `hooks/*.py + .claude/settings.json`

What & Why: The SDK supports two hook surfaces: (a) **file-based hooks** in `.claude/settings.json` that shell out to scripts (production-friendly, language-agnostic, ideal for the audit-redaction pipeline) and (b) **in-process hooks** via `HookMatcher` in `ClaudeAgentOptions(hooks={...})` for validation that needs to deny the call. We use both. **Critical:** the audit redactor only modifies what gets WRITTEN to the audit log, not what flows back to the agent — otherwise tool chaining breaks.

.CLAUDE/SETTINGS.JSON · HOOKS/AUDIT_REDACTED.PY · IN-PROCESS VALIDATION (HOOKMATCHER)

.CLAUDE/SETTINGS.JSON

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "*", "command": "python hooks/log_redacted.py" }
    ],
    "PostToolUse": [
      { "matcher": "*", "command": "python hooks/audit_redacted.py" }
    ]
  }
}
```

HOOKS/AUDIT_REDACTED.PY

```
IMPORT sys, json, re, datetime

MEMBER_RE = re.compile(r"\b[A-Z]\d{6,9}\b")
SSN_RE    = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")
PHONE_RE  = re.compile(r"\b\d{3}-\d{3}-\d{4}\b")
DOB_RE    = re.compile(r"\b\d{4}-\d{2}-\d{2}\b")

FUNCTION _redact(s):
    s = MEMBER_RE.sub("[MEMBER_ID_REDACTED]", s)
    s = SSN_RE.sub("[SSN_REDACTED]", s)
    s = PHONE_RE.sub("[PHONE_REDACTED]", s)
    s = DOB_RE.sub("[DOB_REDACTED]", s)
    RETURN s


    # … (full source in the desktop HTML) …

WITH open("audit_log.jsonl", "a") as f:
    f.write(json.dumps(rec) + "\n")

# CRITICAL: return the ORIGINAL payload so the agent gets unredacted PHI
# for chaining (e.g., member_id needs to flow into the next tool call).
json.dump(payload, sys.stdout)
```

IN-PROCESS VALIDATION (HOOKMATCHER)

```
IMPORT re
FROM claude_agent_sdk import HookMatcher

CPT_RE   = re.compile(r"^[0-9]{5}$|^[A-Z][0-9]{4}$")
NPI_RE   = re.compile(r"^[0-9]{10}$")
ICD10_RE = re.compile(r"^[A-Z]\d{2}(\.\d{1,4})?$")

FUNCTION validate_input(input_data, tool_use_id, context):
    name  = input_data.get("tool_name", "")
    args  = input_data.get("tool_input", {}) or {}
    fail = None
    IF name.endswith("lookup_clinical_criteria") and not CPT_RE.match(args.get("cpt_code", "")):
        fail = f"Invalid CPT: {args.get('cpt_code')!r}"
    ELIF name.endswith("verify_diagnosis_match") and not ICD10_RE.match(args.get("icd10_code", "")):

    # … (full source in the desktop HTML) …


# Then update OPTIONS in agent.py= ClaudeAgentOptions(
    # ... existing fields ...
    hooks={"PreToolUse": [HookMatcher(matcher="mcp__preauth__*",
                                      hooks=[validate_input])]},
)
```

Create the supporting log_redacted.py hook — same stdin/stdout pattern, prints to stderr (so the log line itself doesn't accidentally leak PHI to anything that captures stdout):

HOOKS/LOG_REDACTED.PY

HOOKS/LOG_REDACTED.PY

```
IMPORT sys, json, re, datetime

MEMBER_RE = re.compile(r"\b[A-Z]\d{6,9}\b")
SSN_RE    = re.compile(r"\b\d{3}-\d{2}-\d{4}\b")

payload = json.load(sys.stdin)
ts = datetime.datetime.utcnow().isoformat() + "Z"
name = payload.get("tool_name", "?")
args_str = json.dumps(payload.get("tool_input", {}))
args_str = MEMBER_RE.sub("[MEMBER_ID_REDACTED]", args_str)
args_str = SSN_RE.sub("[SSN_REDACTED]", args_str)
print(f"[{ts}] PRE  {name}({args_str})", file=sys.stderr)
json.dump(payload, sys.stdout)   # CRITICAL: pass-through unchanged
```

Smoke-test each hook script standalone before wiring them up:

```
# Test the log hook (should print redacted to stderr, pass through stdout):
echo '{"tool_name":"get_benefit_summary","tool_input":{"member_id":"A123456","cpt_code":"27447"}}' \
  | python hooks/log_redacted.py

# Test the audit hook (should append a redacted line to audit_log.jsonl):
echo '{"tool_name":"get_benefit_summary","tool_input":{"member_id":"A123456"},"tool_result":{"covered":true}}' \
  | python hooks/audit_redacted.py
cat audit_log.jsonl
```

Then run the agent end-to-end:

```
python -c "import asyncio, agent; print(asyncio.run(agent.run('Is PA-2025-0001 approvable? CPT 27447 ICD M17.11
member A123456 NPI 1234567890 Aetna')))"
```

✅ **Checkpoint — Step 10**

You should see (1) `[timestamp] PRE` log lines on stderr with member_id REDACTED, (2) the agent's APPROVE determination on stdout with the literal member_id correctly used in the rationale, (3) `audit_log.jsonl` has redacted records. Try a 2-character CPT — the in-process `HookMatcher` validator should deny it.

⚠ **PHI REDACTION IS TRICKIER THAN IT LOOKS (STILL)**

The agent NEEDS the unredacted member_id and provider_npi in its working memory to call the next tool. The hook script writes the REDACTED record to `audit_log.jsonl`, but it returns the ORIGINAL payload via stdout. If you redact the stdout payload too, the agent can't chain — `get_benefit_summary` will see `[MEMBER_ID_REDACTED]` as the member_id and 404. Test by running the agent and confirming both: (1) `audit_log.jsonl` has redacted records, (2) the agent still completes all 5 tool calls successfully.

Troubleshooting

- **Hooks don't run** → the SDK looks for `.claude/settings.json` in cwd. Run from the project root.
- **Audit log shows literal member_id** → you forgot to call `_redact()` inside the audit script. Test the redactor standalone first.
- **Agent fails on get_benefit_summary with "no benefit record"** → you redacted the stdout payload too aggressively. Only redact what's WRITTEN to the audit log; pass through the original payload to stdout.
- **In-process validator never fires** → check that `hooks={"PreToolUse": [HookMatcher(...)]}` is a kwarg on `ClaudeAgentOptions` with the matcher pattern `"mcp__preauth__*"`.

Step 11: Sessions — Multi-Case + Fork

`15 min` `sessions.py`

What & Why: Pre-auth reviewers often want what-ifs: "What if the member were on UnitedHealth instead?" `session.fork()` clones the conversation at a point and runs a hypothetical without polluting the official decision history.

SESSIONS.PY

SESSIONS.PY

```
FROM dataclasses import replace
FROM claude_agent_sdk import query, AssistantMessage
FROM agent import OPTIONS

SESSIONS, str] = {}   # case_id → resume token (session_id)

FUNCTION _drive(prompt, options):
    parts, sid = [], None
    FOR msg IN query(prompt=prompt, options=options):
        IF isinstance(msg, AssistantMessage):
            FOR block IN msg.content:
                IF getattr(block, "text", None): parts.append(block.text)
        s = getattr(msg, "session_id", None)
        IF s= s

    # … (full source in the desktop HTML) …

FUNCTION what_if(case_id, hypothetical):

    resume = SESSIONS.get(case_id)
    options = replace(OPTIONS, resume=resume) IF resume else OPTIONS
    text, _ = _drive(hypothetical, options)
    RETURN text
```

Try it — multi-case + fork demo:

```
python -c "
IMPORT asyncio
FROM sessions import chat, what_if

FUNCTION main():
    print('T1, chat('PA-2025-0001', 'Is this approvable? CPT 27447, ICD M17.11, member A123456, NPI 1234567890,
Aetna'))
    print('FORK, what_if('PA-2025-0001', 'What IF the payer were UnitedHealth instead?'))
    print('T2, chat('PA-2025-0001', 'Stick with Aetna. What about a peer-to-peer review?'))

asyncio.run(main())
"
```

✅ **Checkpoint — Step 11**

T1 produces APPROVE under Aetna. FORK shows what would happen with UnitedHealth (different policy, possibly different determination) WITHOUT polluting the main case session. T2 continues from T1 (Aetna context preserved) and addresses the peer-to-peer ask. If T2 references the UnitedHealth hypothetical, your `what_if` is leaking state into `SESSIONS`.

Step 12: Slash Commands

`15 min` `.claude/commands/*.md (3 files)`

What & Why: `/run-preauth PA-2025-0001`, `/test-agent`, `/eval-agent` turn the agent into a one-line workflow inside Claude Code. Reviewers can adjudicate cases without leaving their IDE.

Create 3 files in `.claude/commands/`:

RUN-PREAUTH.MD · TEST-AGENT.MD · EVAL-AGENT.MD

RUN-PREAUTH.MD

```
---
description: Run the pre-auth agent on a case_id from preauth_requests.json
argument-hint: [case_id]
---
Look up case $ARGUMENTS in mock_data/preauth_requests.json. Build the question
string. Run `python -c "import asyncio, agent; print(asyncio.run(agent.run(q)))"`
where q is the question. Print determination, rationale, policy citation,
total tokens, total cost from the ResultMessage emitted by query().
```

TEST-AGENT.MD

```
---
description: Run the unit test suite for the pre-auth agent
---
Run `pytest tests/ -v`. Critical tests: test_phi_not_in_audit (PHI must be
redacted in audit_log.jsonl), test_member_id_passes_to_get_benefit_summary
(unredacted in agent stream), test_approves_pa_2025_0001 (canonical case
should APPROVE with policy CPB-0660 cited).
```

EVAL-AGENT.MD

```
---
description: Run the 15-case evaluation suite
---
Read test_scenarios.json (15 pre-auth cases: 11 APPROVE, 2 DENY, 2 REQUEST_INFO).
For each, call agent.run(), score on: correct decision, policy_id cited, all 5
tools called, tone is appropriate for clinical context. Report per-case score
and overall percentage.
```

✅ **Checkpoint — Step 12**

When you type `/` in Claude Code, the three commands appear in autocomplete. `/run-preauth PA-2025-0001` produces the APPROVE determination. `/test-agent` requires a `tests/` folder — in Iter 3 the spec generates these for you.

Step 13: Deploy via Claude Code

`15 min` `server.py + Dockerfile`

What & Why: Same FastAPI + Docker pattern as Iter 1, but Claude Code writes it.

CLAUDE CODE PROMPT · RESULTING SERVER.PY · DOCKERFILE

CLAUDE CODE PROMPT

```
> Create server.py and Dockerfile. Endpoints: GET /health,
> POST /preauth (single-shot determination), POST /chat (case_id + message).
> Async FastAPI handlers awaiting agent.run() and sessions.chat() (both are
> async coroutines from claude-agent-sdk). python:3.11-slim base, install
> claude-agent-sdk + dependencies, expose 8000. Mount .claude/ into the
> container so settings.json + hook scripts resolve at runtime.
```

RESULTING SERVER.PY

```
FROM fastapi import FastAPI, HTTPException
FROM pydantic import BaseModel
FROM agent import run as agent_run
FROM sessions import chat as session_chat

app = FastAPI(title="Pre-Auth Decision Agent (Iter 2 — SDK)")

CLASS Q(BaseModel): question: str
CLASS C(BaseModel): case_id: str; message: str

FUNCTION health(): return {"status": "ok", "iter": 2}

FUNCTION preauth(q):
    TRY: return {"determination": agent_run(q.question)}
    CATCH: raise HTTPException(500, str(e))

FUNCTION chat_ep(c):
    TRY: return {"answer": session_chat(c.case_id, c.message)}
    CATCH: raise HTTPException(500, str(e))
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
ENV PYTHONUNBUFFERED=1
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8000"]
```

Run locally first, then Docker:

```
uvicorn server:app --reload --port 8000
# In another terminal:
curl localhost:8000/health
curl -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 approvable?"}'

# Then build the container (mounts .claude/ at COPY time):
docker build -t iter2-a .
docker run --rm -p 8000:8000 -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY iter2-a
```

Iteration 2 Complete — End-to-End Verification

You should now have **~14 files** in your project:

```
agent-iter2-sdk/
|— CLAUDE.md
|— agent.py            # query() + 5 @tool functions + create_sdk_mcp_server
|— sessions.py         # chat() + what_if() over SDK resume tokens
|— mock_data/ ×4       # same as Iter 1
|— .claude/
|-   |— settings.json
|-   |— commands/run-preauth.md, test-agent.md, eval-agent.md
|— hooks/
|-   |— log_redacted.py, audit_redacted.py
|— server.py | Dockerfile | requirements.txt
```

Acceptance test — same shape as Iter 1:

```
# 1. Single pre-auth (should APPROVE, same as Iter 1)
curl -s -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 approvable?"}' | python -m json.tool

# 2. Multi-case with SDK resume tokens
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"Is this approvable? CPT 27447, ICD M17.11"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"What about a peer-to-peer review?"}' | python -m json.tool

# 3. PHI verification
docker exec $(docker ps -q --filter ancestor=iter2-a) cat audit_log.jsonl | head -5
# Look for [MEMBER_ID_REDACTED] but the agent's response should still resolve benefits.
```

You pass Iter 2 if: all 3 outputs are functionally equivalent to Iter 1, but you wrote ~120 lines instead of ~250. Iter-2's hooks and SDK do the work the Iter-1 loop did manually.

ITERATION 2 METRICS

- **Files:** ~10 (CLAUDE.md, agent.py, sessions.py, server.py, Dockerfile, .claude/settings.json, hooks/{log_redacted,audit_redacted}.py, slash commands) + 4 mock JSON
- **Lines you wrote:** ~120
- **Time:** ~2 hours
- **Abstractions used:** `claude-agent-sdk` (query / @tool / MCP server / ClaudeAgentOptions / HookMatcher) + `.claude/settings.json` + Claude Code

Debugging in Iteration 2: Hooks + Console Web UI + Langfuse

The SDK abstracts the loop — you cannot drop a print inside it. You debug from the outside.

A. Hooks as Debug Probes

Swap `.claude/settings.json` to a debug variant during diagnostic runs (the script writes redacted info to stderr and passes the original payload through stdout):

HOOKS/DEBUG_REDACTED.PY · .CLAUDE/SETTINGS.DEBUG.JSON

HOOKS/DEBUG_REDACTED.PY

```
IMPORT sys, json, re

MEMBER_RE = re.compile(r"\b[A-Z]\d{6,9}\b")
FUNCTION _r(s): return MEMBER_RE.sub("[MEMBER_ID_REDACTED]", s)

payload = json.load(sys.stdin)
print(f"[HOOK {payload.get('hook_event_name','?')}] "
      f"{payload.get('tool_name','?')}: "
      f"{_r(json.dumps(payload.get('tool_input', payload.get('tool_result'))))[:200]}",
      file=sys.stderr)
json.dump(payload, sys.stdout)   # pass-through unchanged
```

.CLAUDE/SETTINGS.DEBUG.JSON

```
{
  "hooks": {
    "PreToolUse":  [{ "matcher": "*", "command": "python hooks/debug_redacted.py" }],
    "PostToolUse": [{ "matcher": "*", "command": "python hooks/debug_redacted.py" }]
  }
}
```

BETTER than Iter-1 print statements because hooks are modular — symlink `.claude/settings.json` to the debug variant for one run, then back to production. Agent code stays untouched.

B. Anthropic Console Web UI

console.anthropic.com shows every API call. **Worked example:** Agent calls `verify_diagnosis_match` and gets `{"match": false}` for a case that should match. Console → Logs → failed call → tool_use shows Claude sent `icd10_code: "M17"` instead of `"M17.11"`. Fix: strengthen the system prompt to use the FULL ICD-10 code from the request. Re-run; Console shows the fix working in ~3 minutes vs ~15 in Iter 1.

> **PHI IN THE CONSOLE**
>
> Anthropic Console logs include the FULL message content — including any PHI you sent to the API. For real production HIPAA compliance, you would need a BAA with Anthropic and additional measures (e.g., self-hosted deployment via Bedrock or Vertex with VPC isolation). For this capstone, mock data is fine.

C. Langfuse Traces (if instrumented in M19)

Each pre-auth case becomes a waterfall trace: 5 tool calls, each a span. You can compare a 2-second APPROVE case to a 12-second REQUEST_INFO case and see exactly which tool took the time (usually verify_diagnosis_match when the criteria has many indications to check).

D. /run-preauth --verbose

Verbose mode turns on debug hooks for one run. Shows every tool call with redacted args, every hook fire, token count, cost, total time.

SESSION 3

Iteration 3: Spec-Driven

You stop writing code. You write a spec describing what the pre-auth agent should do, then ask Claude Code to build it. ~18 files appear. The spec IS the documentation auditors can read.

`~1 hour` `~100 lines of spec` `~18 generated files` `0 lines of agent code`

Step 14: Write agent-spec.md (12 sections)

`30 min` `agent-spec.md`

What & Why: The spec is your full design doc. Claude Code reads it and produces every file. For pre-auth, sections 4 (System Prompt), 5 (Hooks/PHI), and 11 (Evaluation) are the most important — they encode the compliance rules.

AGENT-SPEC.MD (EXCERPT)

AGENT-SPEC.MD (EXCERPT)

```
# Agent Specification: Pre-Authorization Decision Agent

## 1. Overview
APPROVE / DENY / REQUEST_INFO determinations for medical pre-auth by checking
clinical criteria, diagnosis match, network status, and benefits, then
producing a structured determination with policy citation.

## 2. Configuration
- Model: claude-sonnet-4-6
- Framework: claude-agent-sdk (Python). Tools registered via
  create_sdk_mcp_server. Driver: query() with ClaudeAgentOptions.
- Max turns: 12
- Max tokens per response: 4096


    # … (full source in the desktop HTML) …

|— sessions.py                 # resume-token sessions
|— mock_data/ {preauth_requests, clinical_criteria, provider_directory, benefits}.json
|— hooks/{log_redacted,audit_redacted}.py
|— server.py | Dockerfile | docker-compose.yml | requirements.txt
|— tests/ x5
|— appendix/manual-loop.py     # Iter-1 reference, not for production
```

Step 15: One Command Generates Everything

`10 min` `~18 files appear`

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> /generate-from-spec spec/agent-spec.md

# (or, if /generate-from-spec is not installed:)
> Read spec/agent-spec.md and build the entire project. Create every file
> listed in section 12. Use the `claude-agent-sdk` package (NOT a fictional
> anthropic.Agent class). Tools must be @tool-decorated async functions
> registered via create_sdk_mcp_server. Hooks split between
> .claude/settings.json (file-based redact/audit) and OPTIONS.hooks
> (HookMatcher in-process validation) per section 5. Generate realistic
> mock pre-auth data with 15 cases. Also generate appendix/manual-loop.py
> as an under-the-hood reference.
```

What you should see Claude Code create:

```
Reading spec/agent-spec.md ...
Created agent.py (95 lines — 5 @tool functions + create_sdk_mcp_server + ClaudeAgentOptions)
Created sessions.py (32 lines)
Created hooks/log_redacted.py (15 lines)
Created hooks/audit_redacted.py (24 lines — PHI redaction critical)
Created mock_data/preauth_requests.json (15 cases)
Created mock_data/clinical_criteria.json (5 procedures × 3 payers = 15 policies)
```

```
Created mock_data/provider_directory.json (10 providers)
Created mock_data/benefits.json (30 member-procedure records)
Created server.py (28 lines — async FastAPI)
Created Dockerfile (8 lines)
Created docker-compose.yml (10 lines)
Created requirements.txt (4 lines)
Created CLAUDE.md (40 lines)
Created .claude/settings.json (12 lines)
Created .claude/commands/run-preauth.md, test-agent.md, eval-agent.md
Created tests/test_tools.py, test_agent.py, test_hooks.py, test_sessions.py, test_api.py
Created spec/test_scenarios.json (15 scenarios)
Created appendix/manual-loop.py (~85 lines)
Total: 24 files, ~570 generated lines + 100 lines of spec you wrote.
```

Verify it actually works:

```
# From inside Claude Code:
> /test-agent

# Or from a regular shell:
pytest tests/ -v
```

Expected pytest output (focus on the HIPAA-critical tests):

```
tests/test_tools.py::test_lookup_clinical_criteria_returns_cpb_0660 PASSED
tests/test_tools.py::test_verify_diagnosis_match_m17_11 PASSED
tests/test_tools.py::test_check_network_status_in_network PASSED
tests/test_agent.py::test_pa_2025_0001_returns_approve_with_policy_citation PASSED
tests/test_hooks.py::test_phi_not_in_audit_log PASSED        # ⟵ HIPAA-critical
tests/test_hooks.py::test_member_id_passes_to_get_benefit_summary PASSED  # ⟵ chaining works
tests/test_hooks.py::test_invalid_cpt_denied_by_validator PASSED
tests/test_sessions.py::test_chat_persists_case_context PASSED
tests/test_sessions.py::test_what_if_does_not_pollute_main_case PASSED
tests/test_api.py::test_health_returns_ok PASSED
============================ 10 passed in 14.21s ============================
```

✅ **Checkpoint — Step 15**

All 10 tests pass on the first run. If `test_phi_not_in_audit_log` fails, the generated audit hook forgot the redactor. If `test_member_id_passes_to_get_benefit_summary` fails, the audit hook redacted the stdout payload too aggressively (broke chaining). Both are spec-level fixes — edit section 5 to be more explicit about WHEN to redact.

Troubleshooting

- **Generated hook redacts the stdout payload, breaking tool chaining** → the spec was ambiguous. Section 5 should explicitly say: "Redact ONLY in the persisted audit log; pass the original payload through stdout so the agent can chain."
- **Generated mock data has wrong format** → section 7 of the spec should give a sample record per file. If it just says "preauth_requests.json with 15 cases", Claude Code may invent the schema.
- **Some tests are missing** → section 10 should enumerate every test name. Re-prompt with: "tests/test_hooks.py is missing test_phi_not_in_audit_log per spec section 10."

Step 16: Review & Iterate on the Spec

`15 min` `spec edits + targeted regen`

What & Why: The spec is the source of truth. When you want to add behavior — e.g., a sixth tool `peer_to_peer_review(case_id)` for borderline cases — edit the spec FIRST, then regenerate.

1. Edit `agent-spec.md` section 3 to add `peer_to_peer_review`
2. Edit section 4 to mention "use peer_to_peer_review when criteria are borderline"
3. Edit section 11 to add an eval case

4. In Claude Code: *"I added peer_to_peer_review. Update agent.py, mock_data, add a test, and eval scenario."*

Step 17: Deploy & Compare

`15 min` `same curl, same output`

What & Why: Build, run, curl, compare against Iter 1 and Iter 2. The test of "spec-driven works" is that the agent's *observable behavior* matches the prior iterations.

SHELL

SHELL

```
docker compose up --build -d
docker compose ps   # confirm "Up"

curl localhost:8000/health
# Expected: {"status":"ok","iter":3}

curl -X POST localhost:8000/preauth \
     -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 approvable?"}' | python -m json.tool
```

Cross-iteration diff — the punchline of the whole capstone:

```
# Same query against all three deployments (assumes :8001/8002/8003)
for port in 8001 8002 8003; do
  echo "═══ Iter on :$port ═══"
  curl -s -X POST localhost:$port/preauth -H "Content-Type: application/json" \
       -d '{"question":"Is PA-2025-0001 approvable?"}' \
    | python -c "import json, sys; d = json.load(sys.stdin); print((d.get('determination') or d.get('answer'))
[:300])"
  echo
done
```

Expected: all three return APPROVE with policy CPB-0660 cited. Wording varies; facts match.

Iteration 3 Complete — The Whole Capstone

You have **~24 generated files** + your **~100-line spec**:

```
preauth-agent/
├── spec/agent-spec.md          # YOU wrote this
├── spec/test_scenarios.json    # generated
├── CLAUDE.md | .claude/settings.json | .claude/commands/ ×3   # generated
├── agent.py | sessions.py      # generated (SDK)
├── hooks/log_redacted.py | hooks/audit_redacted.py
├── mock_data/ ×4               # 15 cases, 15 policies, 10 providers, 30 benefits
├── tests/ ×5                    # generated
├── server.py | Dockerfile | docker-compose.yml | requirements.txt
└── appendix/manual-loop.py     # under-the-hood reference
```

Acceptance test — same shape as Iter 1 and Iter 2:

```
# 1. All 10 generated tests pass
pytest tests/ -v
# Expected: 10 passed (especially test_phi_not_in_audit_log)

# 2. End-to-end pre-auth determination
curl -s -X POST localhost:8000/preauth -H "Content-Type: application/json" \
     -d '{"question":"Is PA-2025-0001 approvable?"}' | python -m json.tool

# 3. Multi-case session via SDK resume tokens
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"Is this approvable?"}' | python -m json.tool
curl -s -X POST localhost:8000/chat -H "Content-Type: application/json" \
     -d '{"case_id":"PA-2025-0001","message":"What about peer-to-peer review?"}' | python -m json.tool

# 4. Spec-vs-code drift check
claude
> Read spec/agent-spec.md sections 5 and 10. Compare to hooks/ and tests/.
> Report any drift, especially around PHI redaction.
```

You pass Iter 3 if: (1) all 10 tests pass; (2) PA-2025-0001 returns APPROVE; (3) the second /chat call references prior context; (4) drift check returns "no drift" — especially confirming the audit hook redacts but the agent stream doesn't.

ITERATION 3 METRICS

- **Files:** ~24 (all generated by Claude Code from your spec)
- **Lines you wrote:** ~100 (the spec only)
- **Lines Claude Code wrote:** ~570 (readable, runnable, all tests pass)
- **Time:** ~1 hour (~30 min spec, ~10 min generation, ~20 min review/fix)
- **Abstractions used:** spec format + `/generate-from-spec` + same SDK runtime as Iter 2

Debugging in Iteration 3: Spec Comparison + Tests + Evals

You debug at the spec level. The code is regenerable; the spec is not.

A. Spec vs Code Comparison

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Read agent-spec.md and compare it to the generated agent.py and hooks.py.
> Report any deviations — especially around PHI redaction (section 5).
```

Claude Code reads both and tells you exactly what diverged. The spec is the truth.

B. Test-Driven Debugging

CLAUDE CODE PROMPT

CLAUDE CODE PROMPT

```
> Test test_phi_not_in_audit failed: audit_log.jsonl contains literal "A123456".
> Read agent-spec.md section 5 (Hooks). Read generated hooks.py. Find why
> the post_tool_use audit hook isn't running redaction. Fix it.
```

Claude Code reads spec + code, identifies the missing `_redact()` call, and fixes it.

C. Eval-Driven Debugging

Run `/eval-agent`. Output: case PA-2025-0007 scored 2/3 — "agent APPROVED but provider was out-of-network." Fix: strengthen system prompt section 4 to be explicit about the in-network gate. Regenerate only `agent.py` with the updated prompt. Re-eval → 3/3.

D. Console + Langfuse (same as Iter 2)

Generated code uses the same Agent SDK and hooks as Iter 2; all the same tools work.

THE DEBUGGING EVOLUTION SUMMARY

ITERATION	PRIMARY DEBUG METHOD	SECONDARY	SPEED TO FIX
1 Raw	print() in the loop	Manual message inspection	Slow (find the line)
2 SDK	Hooks + Console Web UI	Langfuse traces	Medium (modular probes)
3 Spec	Spec vs code comparison	Tests + evals + Console	Fast (Claude Code finds it)

The Comparison Table

Same pre-auth agent, same business question. Here is everything that changed:

METRIC	ITER 1: RAW API	ITER 2: AGENT SDK	ITER 3: SPEC-DRIVEN
Lines YOU wrote	~250	~120	~100 (spec only)
Time to build	~3 hours	~2 hours	~1 hour
Agent output	Baseline	Same	Same
PHI redaction	Inline in loop	One `.claude/settings.json` entry + a 20-line stdin/stdout script	5 lines in spec section 5
Multi-case sessions	Manual history dict	SDK sessions	SDK sessions (generated)
Adding a 6th tool (peer-to-peer)	Edit 3 files + add validation	One Claude Code prompt	Update spec, ask to regen
Tests (must pass for HIPAA)	Manual	Claude Code generated	Spec generates them
Documentation for auditors	Separate	CLAUDE.md	Spec IS the doc auditors read
Control over internals	Full	SDK-managed	Least direct (but reviewable)
Understanding needed	Every line	SDK abstractions	Architecture-level
Debugging	print() in loop	Hooks + Console + Langfuse	Spec compare + tests + evals
Onboarding a new clinician-engineer	Read 7 files	Read CLAUDE.md + 8 files	Read 1 spec file

KEY TAKEAWAY

All three produce the same APPROVE/DENY/REQUEST_INFO determinations. The difference is what each iteration teaches you. Iteration 1 teaches WHAT an agent is. Iteration 2 teaches HOW to build efficiently. Iteration 3 teaches HOW production teams work — *especially* in regulated industries where the spec doubles as the compliance document.